Institut Universitaire d'Abidjan



LANGAGE C

K. Olivier PETEY

LANGAGE C

K. Olivier PETEY

SYLLABUS DU COURS

***INTITULE DU COURS : LANGAGE C**

Code : ……

***Type :** CM ou TD ; CM et TD *(barrez la mention inutile)*

***Volume horaire : 30H**……

UE de rattachement : **ALGORITHME ET PROGRAMMATION 1**……

***Niveau du cours : LICENCE 1 MIAGE / GI**……

***Département : INFORMATIQUE**……..

***Semestre : 1**……

***Nombre de crédit : 3** ……

***Nom de l'enseignant : PETEY KRAGBI OLIVIER**……..……

***Contact téléphonique : 77 77 12 75** ……..……

***Email** : peteykol@yahoo.fr.....

***Statut :** Enseignant à l'Université Professionnel d'Entreprise

*Les objectifs

Savoir faire de la programmation procédurale avec le langage C.

*Les objectifs spécifiques

Au l'issue de la formation l'apprenant doit être capable de :

Connaitre les éléments de base du C,

Utiliser des variables et des constantes,

Définir des structures alternatives,

Concevoir des structures itératives,

Gérer les pointeurs, les tableaux

Utiliser les fonctions, les fichiers

Les pré-requis

Avoir une base en algorithme.

……

*Le contenu

CHAPITRE I – NOTIONS DE BASE

I- Introduction

II-Les variables

III-Les constantes

IV- Les operateurs

CHAPITRE II – LES INSTRUCTIONS DE CONTROLE

I-Structures alternatives
If
If … Else
Switch
II- Structure itératives
For
While
Do ….While

CHAPITRE III– LES POINTEURS

I- L'opérateur d'adresse &
II- Les pointeurs

CHAPITRE IV : LES TABLEAUX ET LES CHAINES
I- Les tableaux
II-Les chaines

CHAPITRE V- LES FONCTIONS
I- Fonctions sans passage d'arguments et ne renvoyant rien au programme.
II- Fonctions avec passage d'arguments
III- Le passage de paramètres entre fonctions ou entre fonctions et programme principal

CHAPITRE VI - LES TYPES DE VARIABLES COMPLEXES
I- Les types de variables complexes.
-Les structures
-Les Structures et les tableaux
-Les structures et les pointeurs

CHAPITRE VII - LES FICHIERS

-Généralités
-Manipulation des fichiers

SOMMAIRE

I- Introduction 4
II-Les variables 4
III-Les constantes 4
IV- Les operateurs 4
CHAPITRE 1 : NOTIONS DE BASE DU LANGAGE C 6
I-INTRODUCTION 6
II-LES VARIABLES 12
III-LES CONSTANTES 19
IV-LES OPERATEURS 20
CHAPITRE 2 : LES STRUCTURES DE CONTRÔLES 27
I-LES STRUCTURES ALTERNATIVES 27
CHAPITRE 3 : LES POINTEURS 40
I-L'OPERATEUR ADRESSE & 40
II-LES POINTEURS 41
CHAPITRE 4 : LES TABLEAUX ET LES CHAINES DE CARACTERES 51
I-LES TABLEAUX DE NOMBRES (INT ou FLOAT) 51
II-LES CHAINES DE CARACTERES 58
CHAPITRE 5 : LES FONCTIONS 62
I-FONCTIONS SANS PASSAGE D'ARGUMENTS ET NE RENVOYANT RIEN AU PROGRAMME. 62
II-FONCTION RENVOYANT UNE VALEUR AU PROGRAMME ET SANS PASSAGE D'ARGUMENTS 69
III-FONCTIONS AVEC PASSAGE D'ARGUMENTS 70
CHAPITRE 6 : LES TYPES DE VARIABLES COMPLEXES 77
I. LES DECLARATIONS DE TYPE SYNONYMES: TYPEDEF 77
II. LES STRUCTURES 78
III. STRUCTURES ET TABLEAUX 79
IV. STRUCTURES ET POINTEURS 80
CHAPITRE 7 : LES FICHIERS 81
I-GENERALITES 81
II-MANIPULATION DES FICHIERS 83
BIBLIOGRAPHIE : 88

CHAPITRE 1 : NOTIONS DE BASE DU LANGAGE C

I-INTRODUCTION

Le langage C est un langage évolué et structuré, assez proche du langage machine destiné à des applications de contrôle de processus (gestion d'entrées/sorties, applications temps réel ...). Les compilateurs C possèdent les taux d'expansion les plus faibles de tous les langages évolués (rapport entre la quantité de codes machine générée par le compilateur et la quantité de codes machine générée par l'assembleur et ce pour une même application);

Le langage C possède assez peu d'instructions, il fait par contre appel à des bibliothèques, fournies en plus ou moins grand nombre avec le compilateur.

exemples: math.h : bibliothèque de fonctions mathématiques

stdio.h : bibliothèque d'entrées/sorties standard

Les compilateurs C sont remplacés petit à petit par des compilateurs C++.

Un programme écrit en C est en principe compris par un compilateur C++.

Le cours qui suit est un cours ce langage C écrit dans un contexte C++.

-Fonctions de sorties

Fonction **printf :**

Syntaxe de la fonction printf

printf ("format",[<exp1>,<exp2>,...]);

format correspond aux formats de sortie de chacune des expressions *exp*.

La valeur d'un format peut être :

Format	conversion en	écriture
%d	int	décimale signée
%ld	long int	décimale signée
%u	unsigned int	décimale non signée
%lu	unsigned long int	décimale non signée
%o	unsigned int	octale non signée
%lo	unsigned long int	octale non signée
%x	unsigned int	hexadécimale non signée
%lx	unsigned long int	hexadécimale non signée
%f	double	décimale virgule fixe
%lf	long double	décimale virgule fixe
%e	double	décimale notation exponentielle
%le	long double	décimale notation exponentielle
%g	double	décimale, représentation la plus courte parmi %f et %e
%lg	long double	décimale, représentation la plus courte parmi %lf et %le
%c	unsigned char	caractère
%s	char*	chaîne de caractères

En plus du caractère donnant le type des données, on peut éventuellement préciser certains paramètres du format d'impression, qui sont spécifiés entre le % et le caractère de conversion dans l'ordre suivant :

• largeur minimale du champ d'impression : %10d spécifie qu'au moins 10 caractères seront réservés pour imprimer l'entier. Par défaut, la donnée sera cadrée à droite du champ. Le signe - avant le format signifie que la donnée sera cadrée à gauche du champ (%-10d).

• précision : %.12f signifie qu'un flottant sera imprimé avec 12 chiffres après la virgule. De même %10.2f signifie que l'on réserve 12 caractères (incluant le caractère .) pour imprimer le flottant et que 2 d'entre eux sont destinés aux chiffres après la virgule. Lorsque la précision n'est pas spécifiée, elle correspond par défaut à 6 chiffres après la virgule. Pour une chaîne de caractères, la précision correspond au nombre de caractères imprimés : %30.4s signifie que l'on réserve un champ de 30 caractères pour imprimer la chaîne mais que seulement les 4 premiers caractères seront imprimés (suivis de 26 blancs).

Fonction **putchar :** Affichage d'un caractère

La fonction **putchar** permet d'afficher un caractère:

c étant une variable de type **char**, l'écriture **putchar(c);** est équivalente à **printf("%c\n",c);**

Fonction **puts** :Affichage d'un texte

La fonction **puts** permet d'afficher un texte:

l'écriture **puts("bonjour");** est équivalente à **printf("bonjour\n");**

Il vaut mieux utiliser puts et putchar si cela est possible, ces fonctions, non formatées, sont d'exécution plus rapide, et nécessitent moins de place en mémoire lors de leur chargement.

-Fonctions d'entrée :

Saisie de nombres et de caractères au clavier

La fonction **GETCH**

La fonction getch, appartenant à la bibliothèque conio.h permet la saisie clavier d' un caractère alphanumérique, **sans écho écran**. La saisie s'arrête dès que le caractère a été frappé.

La fonction getch n'est pas définie dans la norme ANSI mais elle peut exister dans la bibliothèque d'autres compilateurs.

On peut utiliser getch de deux façons:

- sans retour de variable au programme:

```
Exemple:    printf("POUR CONTINUER FRAPPER UNE TOUCHE ");
            getch();
```

- avec retour de variable au programme:

```
Exemple:    char alpha;
```

```
printf("ENTRER UN CARACTERE (ATTENTION PAS DE RETURN) ");
alpha = getch();
printf("\nVOICI CE CARACTERE: %c",alpha);
```

Les parenthèses vides de getch() signifient qu'aucun paramètre n'est passé à cette fonction par le programme.

La fonction **SCANF**

La fonction scanf, appartenant à la bibliothèque stdio.h, permet la saisie clavier de n'importe quel type de variable.

Les variables à saisir sont formatées, le nom de la variable est précédé du symbole & désignant l'adresse de la variable (On reverra ce symbole dans le chapitre sur les pointeurs).

La saisie s'arrête avec "RETURN" (c'est à dire LF), les éléments saisis s'affichent à l'écran (**saisie avec écho écran**).

Tous les éléments saisis après un **caractère d'espacement** (espace, tabulation) sont ignorés.

La fonction **scanf** permet de saisir des données au clavier et de les stocker aux adresses spécifiées par les arguments de la fonctions.

scanf("chaîne de contrôle**",**argument1**,...,**argumentN**)**

La chaîne de contrôle indique le format dans lequel les données lues sont converties. Elle ne contient pas d'autres caractères (notamment pas de \n).

Comme pour printf, les conversions de format sont spécifiées par un caractère précédé du signe **%**. Les formats valides pour la fonction scanf diffèrent légèrement de ceux de la fonction printf et sont donnés dans le tableau ci-dessous.

Les données à entrer au clavier doivent être séparées par des blancs ou des <RETURN> sauf s'il s'agit de caractères. On peut toutefois fixer le nombre de caractères de la donnée à lire. Par exemple %3s pour une chaîne de 3 caractères, %10d pour un entier qui s'étend sur 10 chiffres, signe inclus.

Cas de la saisie de plusieurs valeurs :

scanf("%c%d%lf", &un_caractere, &un_entier, &un_double);

Chaque % rencontré dans le format (la chaîne de caractères) est suivi d'un caractère indiquant

la nature de l'information à lire (c pour caractère, d pour entier, etc.). À chaque % doit

correspondre une variable donnée après le format. Les variables sont séparées par des virgules et

sont précédées du signe & .Le & indique que l'on donne l'adresse de la variable de qui permet à scanf de changer la valeur de la variable.

format	type d'objet pointé	représentation de la donnée saisie
%d	int	décimale signée
%hd	short int	décimale signée
%ld	long int	décimale signée
%u	unsigned int	décimale non signée
%hu	unsigned short int	décimale non signée
%lu	unsigned long int	décimale non signée
%o	int	octale
%ho	short int	octale
%lo	long int	octale
%x	int	hexadécimale
%hx	short int	hexadécimale
%lx	long int	hexadécimale
%f	float	flottante virgule fixe
%lf	double	flottante virgule fixe
%Lf	long double	flottante virgule fixe
%e	float	flottante notation exponentielle
%le	double	flottante notation exponentielle
%Le	long double	flottante notation exponentielle
%g	float	flottante virgule fixe ou notation exponentielle
%lg	double	flottante virgule fixe ou notation exponentielle
%Lg	long double	flottante virgule fixe ou notation exponentielle
%c	char	caractère
%s	char*	chaîne de caractères

-Notion de prototype

Les fichiers de type ".h" (conio.h, dos.h, stdio.h etc...), appelés ***fichiers d'en tête*** contiennent la définition des ***prototypes*** des fonctions utilisées dans le programme. Le prototype précise la syntaxe de la fonction: son nom, les paramètres éventuels à passer, la valeur éventuelle retournée au programme.

Grâce aux lignes "#include", le compilateur lit les fichiers de type ".h" et vérifie que la syntaxe de l'appel à la fonction est correcte.

Etapes permettant l'edition, la mise au point, l'execution d'un programme

Le C comme langage compilé

Le langage C est donc un langage compilé (même s'il existe des interpr´eteurs plus ou moins expérimentaux). Ainsi, un programme C est d'ecrit par un fichier texte, appelé fichier source. La compilation se décompose en fait en 4 phases successives :

1. Le traitement par le préprocesseur : le fichier source (portant l'extension .c) est analysé par le préprocesseur qui effectue des transformations purement textuelles (remplacement de chaˆines de caractères, inclusion d'autres fichiers sources, compilation conditionnelle ...).

2. La compilation : la compilation proprement dite traduit le fichier généré par le préprocesseur (d'extension .i) en assembleur, c'est-à-dire en une suite d'instructions du microprocesseur qui utilisent des mnémoniques rendant la lecture possible.

3. L'assemblage : cette opération transforme le code assembleur (extension .s) en un fichier binaire, c'est-à-dire en instructions directement compréhensibles par le processeur. Généralement, la compilation et l'assemblage se font dans la foulée, sauf si l'on spécifie explicitement que l'on veut le code assembleur. Le fichier produit par l'assemblage est appelé fichier objet (et porte l'extension .o). Les fichiers objets correspondant aux librairies

pré-compilées ont pour suffixe .a.

4. L'édition de liens : un programme est souvent séparé en plusieurs fichiers sources, pour

des raisons de clarté mais aussi parce qu'il fait généralement appel à des librairies de fonctions standards déjà écrites. Une fois chaque fichier de code source assemblé, il faut donc lier entre eux les différents fichiers objets. L'édition de liens produit alors un fichier dit exécutable (a.out

par défaut).

Exercice I-1: Editer (EXI_1.C), compiler et exécuter le programme suivant:

```
#include <stdio.h>                /* bibliotheque d'entrees-sorties standard */
#include <conio.h>
void main()
{
puts("BONJOUR");   /* utilisation d'une fonction-bibliotheque */
puts("Pour continuer frapper une touche...");
getch();  /* Attente d'une saisie clavier */
}
```

Le langage C distingue les minuscules, des majuscules. Les mots réservés du langage C doivent être écrits **en minuscules.**

On a introduit dans ce programme la notion d'interface homme/machine (IHM).

- L'utilisateur visualise une information sur l'écran,

- L'utilisateur, par une action sur le clavier, fournit une information au programme.

Modifier le programme comme ci-dessous, puis le tester :

```
#include <stdio.h>                /* bibliotheque d'entrees-sorties standard */
#include <conio.h>
void main()
{

int a, b, somme ;  /* déclaration de 3 variables */
puts("BONJOUR");   /* utilisation d'une fonction-bibliotheque */

a = 10 ; /* affectation* /

b = 50 ;  /* affectation */

somme = (a + b)*2 ;  /* affectation et opérateurs   */

printf("Voici le resultat : %d\n ", somme) ;
puts("Pour continuer frapper une touche...");
getch();  /* Attente d'une saisie clavier */
}
```

Dans ce programme, on introduit 3 nouveaux concepts :

- La notion de déclaration de variables : les variables sont les données que manipulera le programme lors de son exécution. Ces variables sont rangées dans la mémoire vive de l'ordinateur. Elle doivent être déclarées au début du programme.

- La notion d'affectation, symbolisée par le signe =.

- La notion d'opération.

II-LES VARIABLES

Les differents types de variables

1- Les entiers

Le langage C distingue plusieurs types d'entiers:

TYPE	DESCRIPTION	TAILLE MEMOIRE
int	entier standard signé	4 octets: $-2^{31} \leq n \leq 2^{31}-1$

unsigned int	entier positif	4 octets: $0 \leq n \leq 2^{32}$
short	entier court signé	2 octets: $-2^{15} \leq n \leq 2^{15}-1$
unsigned short	entier court non signé	2 octets: $0 \leq\leq n \leq 2^{16}$
char	caractère signé	1 octet : $-2^{7} \leq\leq n \leq 2^{7}-1$
unsigned char	caractère non signé	1 octet : $0 \leq n \leq 2^{8}$

Numération: En décimal les nombres s'écrivent tels que, précédés de 0x en hexadécimal.

exemple: 127 en décimal s'écrit 0x7f en hexadécimal.

Remarque: En langage C, le type **char** est un cas particulier du type entier:

un caractère est un entier de 8 bits

Exemples:

Les caractères alphanumériques s'écrivent entre ‘ ‘

Le **caractère** 'b' a pour valeur 98 (son code ASCII).

Le **caractère** 22 a pour valeur 22.

Le **caractère** 127 a pour valeur 127.

Le **caractère** 257 a pour valeur 1 (ce nombre s'écrit sur 9 bits, le bit de poids fort est perdu).

Quelques constantes caractères:

CARACTERE	VALEUR (code ASCII)	NOM ASCII

'\n'	interligne	0x0a	LF
'\t'	tabulation horizontale	0x09	HT
'\v'	tabulation verticale	0x0b	VT
'\r'	retour charriot	0x0d	CR
'\f'	saut de page	0x0c	FF
'\\'	backslash	0x5c	\
'\"	cote	0x2c	'
'\"'	guillemets	0x22	"

Modifier ainsi le programme et le tester :

```
#include <stdio.h>                /* bibliotheque d'entrees-sorties standard */
#include <conio.h>
void main()
{
int a, b, calcul ;  /* déclaration de 3 variables */
char u, v;
puts("BONJOUR");   /* utilisation d'une fonction-bibliotheque */
a = 10 ; /* affectation* /
b = 50 ;  /* affectation */
u = 65 ;
v = 'A' ;
calcul = (a + b)*2 ;  /* affectation et opérateurs   */
printf("Voici le resultat : %d\n", calcul) ;
printf("1er affichage de u : %d\n ",u) ;
printf(" 2ème affichage de u : %c\n ",u) ;
 printf(" 1er affichage de v: %d\n"v) ;
printf(" 2éme  affichage de v: %c\n ",v) ;
puts("Pour continuer frapper une touche...");
getch();  /* Attente d'une saisie clavier */
}
```

2- Les réels

Un réel est composé - d'un signe - d'une mantisse - d'un exposant

Un nombre de bits est réservé en mémoire pour chaque élément.

Le langage C distingue 2 types de réels:

TYPE	DESCRIPTION	TAILLE MEMOIRE
float	réel standard	4 octets
double	réel double précision	8 octets

Les initialisations

Le langage C permet l'initialisation des variables dans la zône des déclarations:

char c; est équivalent à char c = 'A';

c = 'A';

int i; est équivalent à int i = 50;

i = 50;

Cette règle s'applique à tous les nombres, char, int, float ...

Sorties de nombres ou de texte a l'ecran

La fonction printf

Ce n'est pas une instruction du langage C, mais une fonction de la bibliothèque stdio.h.

Exemple: affichage d'un texte:

printf("BONJOUR"); /* pas de retour à la ligne du curseur apres l'affichage, */

printf("BONJOUR\n"); /* affichage du texte, puis retour à la ligne du curseur. */

Exercice I-2: Tester le programme suivant et conclure.

```
#include <stdio.h>
#include <conio.h>

void main()
{
printf("BONJOUR ");
printf("IL FAIT BEAU\n");  /* equivalent à puts("BONJOUR";  */
printf("BONNES VACANCES");
puts("Pour continuer frapper une touche...");
getch();  /* Attente d'une saisie clavier */
}
```

La fonction printf exige l'utilisation de formats de sortie, avec la structure suivante:

printf("%format",nom_de_variable);

Exercice I-3: Affichage d'une variable de type **char:**

Tester le programe suivant et conclure.

Dans un deuxième temps, le modifier ce programme pour améliorer l'interface utilisateur.

```
#include <stdio.h>
#include <conio.h>
void main()
```

```
{
char c;
c =66;                  /* c est le caractere alphanumerique A */
printf("%d\n",c);       /* affichage du code ASCII en decimal */
                              /* et retour … à la ligne */
printf("%o\n",c);       /* affichage du code ASCII en base huit
                                 /* et retour … à la ligne */
printf("%x\n",c);       /* affichage du code ASCII en hexadecimal
                                /* et retour … à la ligne */
printf("%c\n",c);       /* affichage du caractère */
                                /* et retour à la ligne */
puts("Pour continuer frapper une touche...");
getch();  /* Attente d'une saisie clavier */
}
```

Exercice I-4: Affichage multiple de structure:

printf("format1 format2 .... formatn",variable1,variable2, ...,variablen);

Tester le programe suivant et conclure:

```
#include <stdio.h>
#include <conio.h>
void main()
{
char c;
c ='A'; /* c est le caractere alphanumerique A */
printf("decimal = %d  ASCII = %c\n",c,c);
puts("Pour continuer frapper une touche...");
getch();  /* Attente d'une saisie clavier */
}
```

Formats de sortie pour les entiers:

%d affichage en décimal (entiers de type int),

%x affichage en hexadécimal (entiers de type int),

%u affichage en décimal (entiers de type unsigned int),

D'autres formats existent, consulter une documentation constructeur.

Exercice I-5:

a et b sont des entiers, a = -21430 b = 4782, calculer et afficher a+b, a-b, a*b, a/b, a%b en format décimal, et en soignant l'interface homme/machine.

a/b donne le quotient de la division, a%b donne le reste de la division.

Exercice I-6:

Que va-t-il se produire, à l'affichage, lors de l'exécution du programme suivant ?

```
#include <stdio.h>  /* ex I_6 */

#include <conio.h>
void main()
{
char a = 0x80;
unsigned char b = 0x80;

clrscr();
printf("a en decimal vaut: %d\n",a);
printf("b en decimal vaut: %d\n",b);
puts("Pour continuer frapper une touche...");
getch();  /* Attente d'une saisie clavier */
}
```

Exercice I-7:

En C standard, la taille des entiers est de 32 bits;

Que va-t-il se passer, à l'affichage, lors de l'exécution du programme suivant ?

```
#include <stdio.h>  /* ex I_7.C */
#include <conio.h>

void main()
{
int a = 12345000, b = 60000000, produit;

produit=a*b;
printf("a*b = %d\n",produit);
printf("a*b (en hexa) = %x\n",produit);
puts("Pour continuer frapper une touche...");
getch();  /* Attente d'une saisie clavier */
}
```

Format de sortie pour les réels: %f

Exercice I-8:

a et b sont des réels, a = -21,43 b = 4,782, calculer et afficher a+b, a-b, a*b, a/b, en soignant l'interface homme/machine.

III-LES CONSTANTES

Le langage C autorise 2 méthodes pour définir des constantes.

1ere méthode: **déclaration** d'une variable, dont la valeur sera constante pour tout le programme:

Exemple:

```
void main()
{
const float PI = 3.14159;
float perimetre,rayon = 8.7;
perimetre =  2*rayon*PI;
....
}
```

Dans ce cas, le compilateur réserve de la place en mémoire (ici 4 octets), pour la variable pi, mais dont on ne peut changer la valeur.

2eme méthode: **définition d'un symbole** à l'aide de la directive de compilation **#define.**

Exemple:

```
#define PI = 3.14159;
void main()
{
float perimetre,rayon = 8.7;
perimetre =  2*rayon*PI;
....
}
```

Le compilateur ne réserve pas de place en mémoire. Les constantes déclarées par #define s'écrivent traditionnellement en majuscules, mais ce n'est pas une obligation.

IV-LES OPERATEURS

- **Opérateurs arithmétiques**

Opérateur	Traduction	Exemple	Résultat
+	Addition	x + y	l'addition de x et y
-	Soustraction	x - y	la soustraction de x et y
*	Produit	x * y	la multiplication de x et y
/	Division	x / y	le quotient de x et y
%	Reste	x % y	Reste de la division euclidienne de x par y
+(unaire)	Signe positif	+x	la valeur de x
-(unaire)	Signe négatif	-x	la négation arithmétique de x
++(unaire)	Incrément	x++ ou ++x	x est incrémenté ($x = x + 1$). L'opérateur préfixe ++x (resp. suffixe x++) incrémente x avant (resp. après) de l'évaluer
--(unaire)	Decrément	x-- ou --x	x est décrementé ($x = x - 1$). L'opérateur préfixe --x (resp. suffixe x--) décrémente x avant (resp. après) de l'évaluer

– Concernant l'incrémentation pré/postfixe, voici un petit exemple pour bien comprendre :

Supposons que la valeur de N soit égale à 5 :

– Incrémentation postfixe : X = N++ ; Résultat : N = 6 et X = 5

– Incrémentation préfixe : X = ++N ; Résultat : N = 6 et X = 6

Exemple particulier:

```
char c,d;
c = 'G';
d = c+'a'-'A';
```

Les caractères sont des entiers sur 8 bits, on peut donc effectuer des opérations. Sur cet exemple, on transforme la lettre majuscule G en la lettre minuscule g.

- **Opérateurs d'affectation**

Opérateur	Traduction	Exemple	Résultat
`=`	affectation simple	`x = y`	assigne la valeur de y à x
`(op)=`	affectation composée	`x += y`	`x (op)=y` est équivalent à `x = x (op) y`

Dans l'écriture x(op)=y, la variable x n'est évaluée qu'une seule fois. (op) est un opérateur arithmétique ou de manipulation de bits Les op´erateurs d'affectation

composés sont donc :

+= -= *= /= %= &= ^= |= <<= >>=

- **Opérateurs relationnels**

Toute comparaison est une expression de type int qui renvoie la valeur 0 (false) ou 1 (true). Il faut que les opérandes soient du même type arithmétique (ou des pointeurs sur des objets de même type).

Opérateur	Traduction	Exemple	Résultat
<	inférieur	`x < y`	1 si x est inférieur à y
<=	inférieur ou égal	`x <= y`	1 si x est inférieur ou égal à y
>	supérieur	`x > y`	1 si x est supérieur à y
>=	supérieur ou égal	`x >= y`	1 si x est supérieur ou égal à y
==	égalité	`x == y`	1 si x est égal à y
!=	inégalité	`x != y`	1 si x est différent de y

- **Opérateurs logiques**

Opérateur	Traduction	Exemple	Résultat
&&	ET logique	`x && y`	1 si x et y sont différents de 0
||	OU logique	`x || y`	1 si x et/ou y sont différents de 0
!	NON logique	`!x`	1 si x est égal à 0. Dans tous les autres cas, 0 est renvoyé.

Les opérandes des opérateurs logiques peuvent être n'importe quel scalaire (i.e arithmétique ou pointeur). Toute valeur différente de 0 est interprétée comme vraie (et 0 correspond à 'faux'). Comme pour les expressions relationnelles, les expressions logiques renvoient une valeur entière (0=false ; 1=true).

Remarque : les opérateurs && et || évaluent les opérandes de gauche à droite et le résultat est connu dès l'opérande de gauche. Ainsi, l'opérande de droite n'est évaluée que si celle de gauche est vraie dans le cas de l'opérateur && (respectivement fausse dans le cas de l'opérateur ||). Par exemple, dans l'expression (i < max) && (f(14) == 1), la fonction f n'est appelée que si i < max.

- **Opérateurs bit à bit**

Opérateur	Traduction	Exemple	Résultat (pour chaque position de bit)
&	ET bit à bit	x & y	1 si les bits de x et y valent 1
|	OU bit à bit	x | y	1 si le bit de x et/ou de y vaut 1
^	XOR bit à bit	x ^ y	1 si le bit de x ou de y vaut 1
~	NON bit à bit	~x	1 si le bit de x est 0
<<	décalage à gauche	x << y	décale chaque bit de x de y positions vers la gauche
>>	sécalage à droite	x >> y	décale chaque bit de x de y positions vers la droite

Les opérateurs bits à bits n'opèrent que sur des entiers. Les opérandes sont interprétées bits par bits (le bit 1 correspondant à une valeur vraie, 0 est considéré comme une valeur fausse). Quelques exemples pour bien comprendre (chaque opérande est fournie sous forme décimale et binaire) :

x		y		Opération	Résultat	
14	1110	9	1001	x & y	8	1000
14	1110	9	1001	x | y	15	1111
14	1110	9	1001	x ^ y	7	0111
14	1110			~x	1	0001
14	1110	2	0010	x << y	56	111000
14	1110	2	0010	x >> y	3	11

Exemples: p = n « 3; /* p est égale à n décalé de 3 bits à gauche */

p = n » 3; /* p est égale à n décalé de 3 bits à droite */

- **Opérateurs d'accès à la mémoire**

L'opérande de l'opérateur d'adresse & doit être une expression qui désigne un objet ou une expression. Ainsi, &x renvoie l'adresse mémoire de x et est donc un pointeur vers x. Plus de détails dans le chapitre dédié aux pointeurs.

Op.	Traduction	Exemple	Résultat
`&`	Adresse de	`&x`	l'adresse mémoire de x
`*`	Indirection	`*p`	l'objet (ou la fonction) pointée par p
`[ ]`	Elément de tableau	`t[i]`	L'équivalent de `*(x+i)`, l'élément d'indice i dans le tableau t
`.`	Membre d'une structure ou d'une union	`s.x`	le membre x dans la structure ou l'union s
`->`	Membre d'une structure ou d'une union	`p->x`	le membre x dans la structure ou l'union pointée par p

- **Autres opérateurs**

Op.	Traduction	Exemple	Résultat
`()`	Appel de fonction	`f(x,y)`	Exécute la fonction f avec les arguments x et y
`(type)`	cast	`(long)x`	la valeur de x avec le type spécifié
`sizeof`	taille en bits	`sizeof(x)`	nombre de bits occupé par x
`? :`	Evaluation conditionnelle	`x?:y:z`	si x est différent de 0, alors y sinon z
`,`	séquencement	`x,y`	Evalue x puis y

On y retrouve notamment l'appel de fonction et la conversion de type (qui ne s'applique qu'aux types scalaires et qui sera abordée plus en détail.

L'opérateur **sizeof(type)** renvoie le nombre d'octets réservés en mémoire pour chaque type d'objet.

Exemple: n = sizeof(char); /* n vaut 1 */

Exercice I-9: n est un entier (n = 0x1234567a), p est un entier (p = 4). Ecrire un programme qui met à 0 les p bits de poids faibles de n.

Exercice I-10: Quels nombres va renvoyer le programme suivant ?

```
#include <stdio.h>
#include <conio.h>

void main()
{

printf("TAILLE D'UN CARACTERE:%d\n",sizeof(char));
printf("TAILLE D'UN ENTIER:%d\n",sizeof(int));
```

```
printf("TAILLE D'UN REEL:%d\n",sizeof(float));
printf("TAILLE D'UN DOUBLE:%d\n",sizeof(double));
puts("Pour continuer frapper une touche...");
getch();  /* Attente d'une saisie clavier */
}
```

-Incrementation - decrementation

Le langage C autorise des écritures simplifiées pour l'incrémentation et la décrémentation de variables:

i = i+1; est équivalent à i++;

i = i-1; est équivalent à i--;

-Operateurs combines

Le langage C autorise des écritures simplifiées lorsqu'une même variable est utilisée de chaque côté du signe = d'une affectation. Ces écritures sont à éviter lorsque l'on débute l'étude du langage C car elles nuisent à la lisibilité du programme.

a = a+b; est équivalent à a+= b;

a = a-b; est équivalent à a-= b;

a = a & b; est équivalent à a&= b;

LES CONVERSIONS DE TYPES

Le langage C permet d'effectuer des opérations de conversion de type: On utilise pour cela l'opérateur de "cast" ().

Exemple et exercice I-11:

```
#include <stdio.h>
#include <conio.h>
void main()
{
int i=0x1234,j;
char d,e;
float r=89.67,s;
j = (int)r;
s = (float)i;
d = (char)i;
e = (char)r;
printf("Conversion float -> int: %5.2f -> %d\n",r,j);
printf("Conversion int -> float: %d -> %5.2f\n",i,s);
printf("Conversion int -> char: %x -> %x\n",i,d);
printf("Conversion float -> char: %5.2f -> %d\n",r,e);
printf("Pour sortir frapper une touche ");getch();
}
```

POINTS IMPORTANTS A RETENIR

- Notions de base du langage C
-Déclarations et utilisation des variables
- Déclarations et utilisation des constantes
- Utilisations des operateurs

CHAPITRE 2 : LES STRUCTURES DE CONTRÔLES

I-LES STRUCTURES ALTERNATIVES

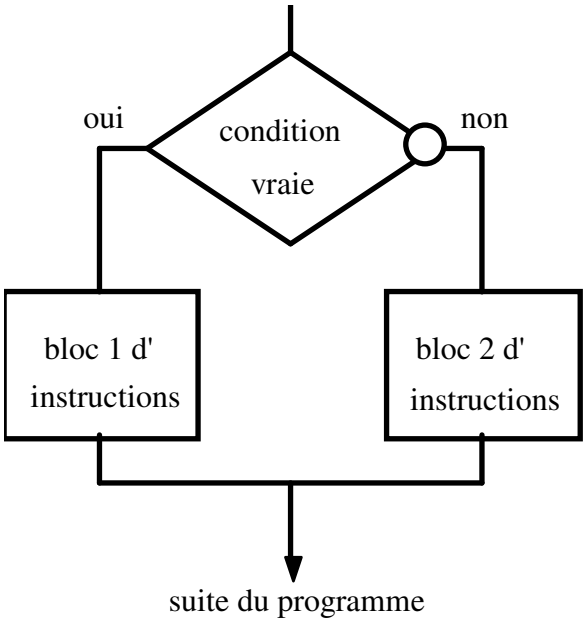
-L'instruction si ... Alors ... Sinon ...

Il s'agit de l'instruction: **si (expression conditionnelle vraie)**

alors {BLOC 1 D'INSTRUCTIONS}

sinon {BLOC 2 D'INSTRUCTIONS}

Organigramme:



Syntaxe en C: **if (expression)**

{

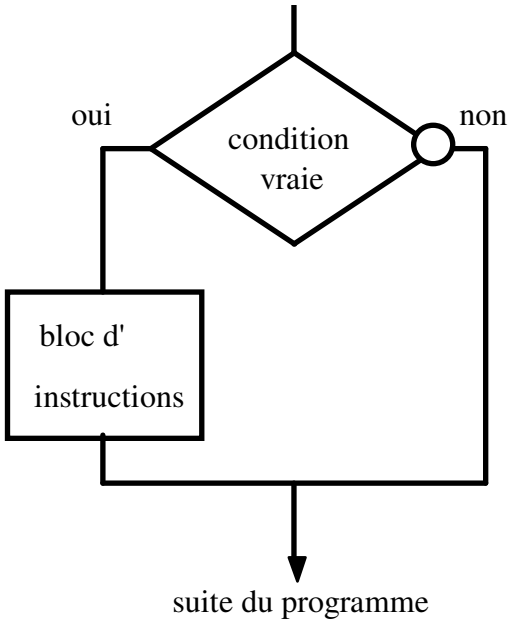
.....; **/* bloc 1 d'instructions */**

.....;

```
.....;
}
else
{
.....;                    /* bloc 2 d'instructions */
.....;
.....;
}
```

Le bloc "sinon" est optionnel: **si (expression vraie)**

alors {BLOC D'INSTRUCTIONS}



Syntaxe en C: **if (expression)**

```
{
.....;                    /* bloc d'instructions */
.....;
.....;
}
```

Remarque: les {} ne sont pas nécessaires lorsque les blocs ne comportent qu'une seule instruction.

-Les operateurs logiques

test d'égalité: **if (a==b)** **" si a égal b "**

test de non égalité: **if (a!=b)** **" si a différent de b "**

tests de relation d'ordre: **if (a<b)** **if (a<=b)** **if (a>b)** **if (a>=b)**

test de ET LOGIQUE: **if ((expression1) && (expression2))**

" si l'expression1 ET l'expression2 sont vraies "

test de OU LOGIQUE **if ((expression1) || (expression2))**

" si l'expression1 OU l'expression2 est vraie " test de NON LOGIQUE **if (!(expression1))**

" si l'expression1 est fausse "

Toutes les combinaisons sont possibles entre ces tests.

Exercice III-1: L'utilisateur saisit un caractère, le programme teste s'il s'agit d'une lettre majuscule, si oui il renvoie cette lettre en minuscule, sinon il renvoie un message d'erreur.

Le langage C admet des écritures contractées dans les expressions de test:

```
char reponse;                          est équivalent à char reponse;
printf("Voulez-vous jouer ?");                 printf("Voulez-vous jouer ?");
reponse = getchar();
if(reponse == 'o')                             if((reponse = getchar()) =='o')
printf("BONJOUR\n");                           printf("BONJOUR\n");
```

else printf("TANT-PIS"\n); else printf("TANT-PIS"\n);

-L'instruction au cas ou ... Faire ...

L'instruction switch permet des choix multiples **uniquement sur des entiers (int) ou des caractères (char).**

Syntaxe:

switch(variable de type char ou int) au cas où la variable vaut:

{

case valeur1: .....; - cette valeur1: executer ce bloc d'instructions.

.....;

break;

valeur2:.....; - cette valeur2: executer ce bloc d'instructions.

.....;

break;

.

. etc ...

.

default: .....; - aucune des valeurs précédentes: executer ce bloc

.....; d'instructions, pas de "break" ici.

}

le bloc "default" n'est pas obligatoire.

L'instruction switch correspond à une cascade d'instructions if ...else

Exemple:

Cette instruction est commode pour fabriquer des "menus":

```
char choix;
printf("LISTE PAR GROUPE TAPER 1\n");
printf("LISTE ALPHABETIQUE TAPER 2\n");
printf("POUR SORTIR TAPER S\n");
printf("\nVOTRE CHOIX: ");
choix = getchar();
switch(choix)
{
case '1': .......;
          .......;
          break;

case '2':  ......;
           ......;
           break;

case 'S': printf("\nFIN DU PROGRAMME ....");
           break;

default;  printf("\nCE CHOIX N'EST PAS PREVU ");  /* pas de break ici */
}
```

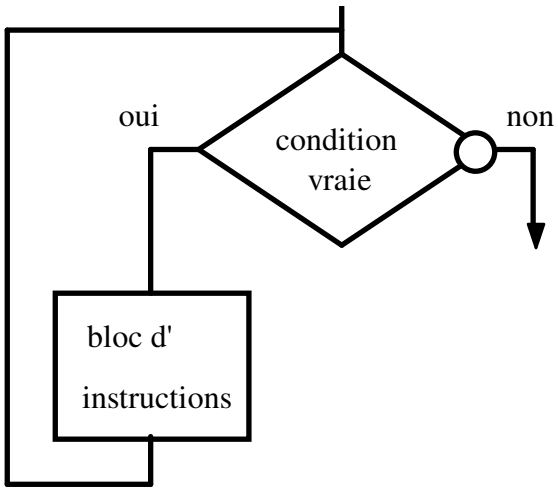
II-STRUCTURES ITERATIVES

-La boucle tant que ... Faire ...

Il s'agit de l'instruction: **tant que (expression vraie)**

faire{BLOC D'INSTRUCTIONS}

Organigramme:



Syntaxe en C: **while (expression)**

{

.....; **/* bloc d'instructions */**

.....;

.....;

}

Le test se fait **d'abord,** le bloc d'instructions n'est pas forcément exécuté.

Remarque: les {} ne sont pas nécessaires lorsque le bloc ne comporte qu'une seule instruction.

Remarque: On peut rencontrer la construction suivante: **while (expression);** terminée par un ; et sans la présence du bloc d'instructions. Cette construction signifie: **"tant que l'expression est vraie attendre".**

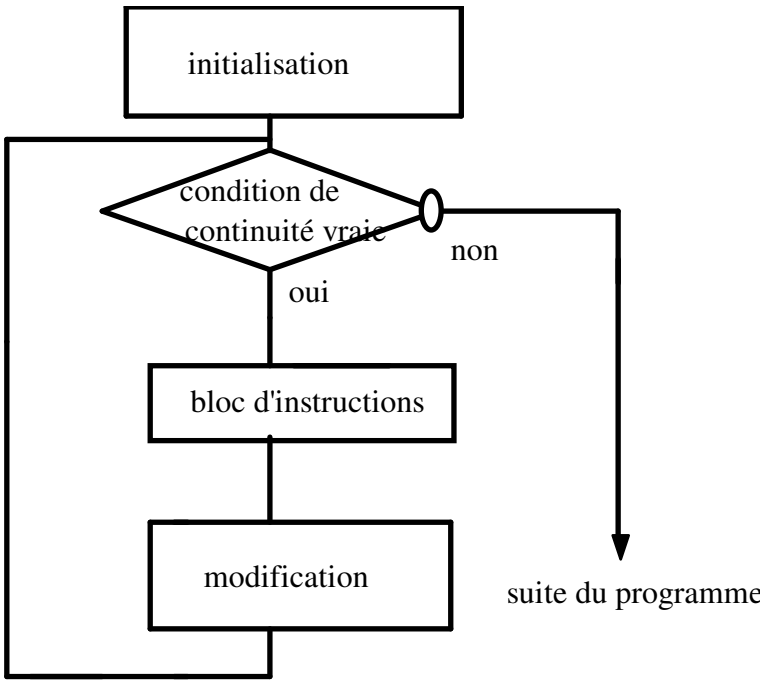
-**L'instruction pour ...**

Il s'agit de l'instruction:

pour (initialisation; condition de continuité vraie;modification)

{BLOC D'INSTRUCTIONS}

Organigramme:



Syntaxe en C:

for(initialisation ; condition de continuité ; modification)

```
{
.....;                    /* bloc d'instructions */
.....;
.....;
}
```

Remarques:

Les { } ne sont pas nécessaires lorsque le bloc ne comporte qu'une seule instruction.

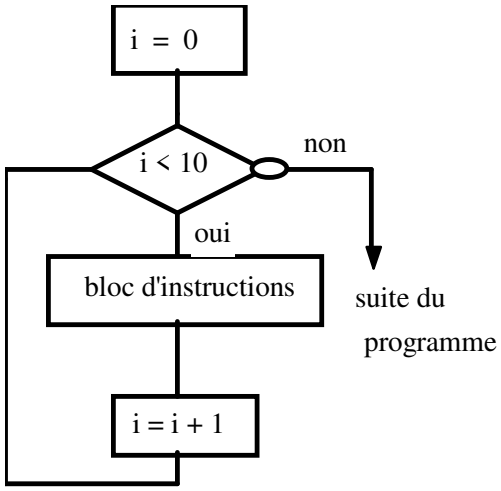
Les 3 instructions du for ne portent pas forcément sur la même variable.

Une instruction peut être omise, mais pas les ;

Exemples:

```
for(i = 0 ; i<10 ; i++)
{
.....;                  /* bloc d'instructions */
.....;
.....;
}
```

correspond à l'organigramme suivant:



La boucle

```
for(;;)
{
.....;                  /* bloc d'instructions */
.....;
.....;
}
```

est une boucle infinie (répétition infinie du bloc d'instructions).

Utilisation de variables différentes:

```
resultat = 0;
for(i = 0 ; resultat<30 ; i++)
        {
        .....;                  /* bloc d'instructions */
        .....;
        .....;
        resultat = resultat + 2*i;
        }
```

Exercice III-2:

Saisir un entier, calculer n!

Utiliser une boucle while puis une boucle for.

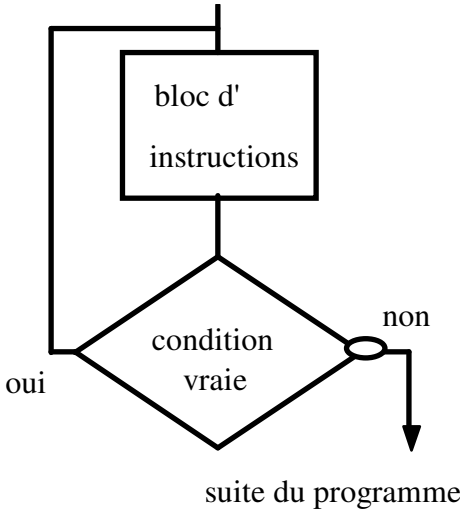
Quelle est la plus grande valeur possible de n, si n est déclaré int, puis unsigned ?

-L'instruction repeter ... Tant que ...

Il s'agit de l'instruction: **répéter{BLOC D'INSTRUCTIONS}**

tant que (expression vraie)

Organigramme



Syntaxe en C: **do**

{

.....; **/* bloc d'instructions */**

.....;

.....;

}

while (expression);

Le test se faisant **après,** le bloc est exécuté au moins une fois.

Remarque: les {} ne sont pas nécessaires lorsque le bloc ne comporte qu'une seule instruction.

-**Complément sur les tests**

En langage C, **une expression nulle de type entier (int) est fausse, une expression non nulle de type entier (int) est vraie.**

Exemples:

int a,b,c,delta; est équivalent à **int a,b,c,delta;**

delta = b*b-4*a*c; **delta = b*b-4*a*c;**

if(delta != 0)

{ ..... }

if(delta)

{ ..... }

int a,b,c,delta;
delta = b*b-4*a*c;
if(delta == 0)
{ ..... }

est équivalent à

int a,b,c,delta;
delta = b*b-4*a*c;
if(!delta)
{.....}

- **Les instruction break ET continue (sauts associés aux boucles)**

L'instruction **break**

L'instruction break fournit un moyen de sortir (passer à l'instruction suivant la boucle). Elle ne permet cependant que de sortir de la boucle la plus interne.

Exemple:

```
int i, tableau[10];
for (i = 0; i < 10; i++)
{
    if (tableau[i] == 0) break;        /* sortie du for si tableau[i] == 0 * /
                    tableau[i] = tableau[i] + 1 ;
}
```

L'instruction **continue**

l'instruction continue fait débuter l'itération suivante de la boucle, en sautant la séquence d'instructions comprise entre le continue et la fin de la boucle.

Exemple:

```
int i, tableau[10];
for (i = 0; i < 10; i++)
{
    if (tableau[i] == 0) continue;    /* on passe à l'élément suivant (si i < 10) */
    tableau[i] = tableau[i] + 1 ;
}
```

L'instruction **return**

return [expression];

L'instruction *return* permet de sortir d'une fonction.

Exemples:

return; ***/* la fonction s'interrompt et rend la main à la fonction appelante */***

return expression ***/* renvoie la valeur de l'expression à la fonction appelante */***

L'instruction **goto**

goto label;

L'instruction goto est associée à un identificateur symbolique spécifiant l'endroit où doit être effectué le branchement inconditionnel.

Exemple:

…

goto label;

…

label: ...

Son emploi doit être évité, car contraire à l'esprit de programmation structurée.

EXERCICES RECAPITULATIFS

Exercice III_3: résoudre $ax^2 + bx + c = 0$.

Exercice III_4: Saisir une suite de caractères, compter et afficher le nombre de lettres e et d'espaces. Utiliser les propriétés du tampon.

Exercice III_5: La fonction kbhit appartient à la bibiothèque conio.h. Une fonction équivalente peut exister avec d'autres compilateurs. La fonction kbhit teste si un caractère a été frappé au clavier. Tant que ce n'est pas vrai kbhit renvoie 0 (ceci signifie que la valeur de la fonction kbhit est 0).

Exemple: **while(kbhit() == 0)** /*tant qu'aucun caractère n'a été frappé exécuter la boucle*/

{ ..... }

Cette écriture est équivalente à:

while(!kbhit()); /* tant que kbhit est faux, exécuter la boucle */

{ ..... }

Ecrire un programme qui affiche le carré des entiers 1, 2, 3 ....., toutes les 500 ms tant qu'aucun caractère n'a été frappé au clavier. Générer la temporisation à l'aide d'une boucle for et d'un décompteur.

POINTS IMPORTANTS A RETENIR

- Utilisation des structures alternatives
If
If … Else
Switch
- Utilisation des structures itératives
For
While
Do ….While
-Utilisation de Break, continue, goto

CHAPITRE 3 : LES POINTEURS

L'étude des pointeurs montre l'adaptation du langage C à la conduite de processus. On verra dans ce chapitre et les suivants la puissance de cette notion par ailleurs de concept simple pour un informaticien industriel.

I-L'OPERATEUR ADRESSE &

I.1 Définition

L'opérateur adresse & retourne l'adresse d'une variable en mémoire.

Exemple:
```
int i = 8;
printf("VOICI i: %d\n",i);
printf("VOICI SON ADRESSE EN HEXADECIMAL: %p\n",&i);
```

On remarque que le format d'une adresse est **%p** (hexadécimal) ou **%d** (décimal) dans printf.

Exercice V_1: Exécuter l'exemple précédent, et indiquer les cases-mémoire occupées par la variable i.

I.1 Adresse et valeur d'un objet

On appelle *Lvalue* (left value) tout objet pouvant être placé à gauche d'un opérateur d'affectation.
Une *Lvalue* est caractérisée par :

- son adresse, c'est-à-dire l'adresse-mémoire à partir de laquelle l'objet est stocké ;
- sa valeur, c'est-à-dire ce qui est stocké à cette adresse.

Dans l'exemple,

```
int i, j;
i = 3;
j = i;
```

Si le compilateur a placé la variable i à l'adresse 4831836000 en mémoire, et la variable j à l'adresse 4831836004, on a

objet	adresse	valeur
i	4831836000	3
j	4831836004	3

Deux variables différentes ont des adresses différentes. L'affectation i = j; n'opère que sur les valeurs des variables. Les variables i et j étant de type int, elles sont stockées sur 4 octets. Ainsi la valeur de i est stockée sur les octets d'adresse 4831836000 à 4831836003.

L'adresse d'un objet étant un numéro d'octet en mémoire, il s'agit d'un entier quelque soit le type de l'objet considéré. Le format interne de cet entier (16 bits, 32 bits ou 64 bits) dépend des architectures. Sur un DEC alpha, par exemple, une adresse a toujours le format d'un entier long (64 bits).

L'opérateur & permet d'accéder à l'adresse d'une variable. Toutefois &i n'est pas une Lvalue mais une constante : on ne peut pas faire figurer &i à gauche d'un opérateur d'affectation. Pour pouvoir manipuler des adresses, on doit donc recourir un nouveau type d'objets, les pointeurs.

II-LES POINTEURS

Toute variable manipulée dans un programme est stockée quelque part en mémoire centrale. Cette mémoire est constituée d'octets qui sont identifiés de manière univoque par un numéro qu'on appelle *adresse*. Pour retrouver une variable, il suffit donc de connaître l'adresse de l'octet où elle est stockée (ou, s'il s'agit d'une variable qui recouvre plusieurs octets contigus, l'adresse du premier de ces octets). Pour des raisons évidentes de lisibilité, on désigne souvent les variables par des identificateurs, et non par leur adresse. C'est le compilateur qui fait alors le lien entre l'identificateur d'une variable et son adresse en mémoire. Toutefois, il est parfois très pratique de manipuler directement une variable par son adresse.

Définition: Un pointeur est une adresse mémoire. On dit que le pointeur pointe sur cette adresse.

-**Declaration des pointeurs**

Une variable de type pointeur se déclare à l'aide de l'objet pointé précédé du symbole *** (opérateur d'indirection).**

Exemple: char *pc; pc est un pointeur pointant sur un objet de type char

int *pi; pi est un pointeur pointant sur un objet de type int

float *pr; pr est un pointeur pointant sur un objet de type float

L'opérateur * désigne en fait le contenu de l'adresse.

Exemple:
```
char *pc;
*pc = 34 ;
printf("CONTENU DE LA CASE MEMOIRE: %c\n",*pc);
printf("VALEUR DE L'ADRESSE EN HEXADECIMAL: %p\n",pc);
```

II.1 Notion de pointeur

Un *pointeur* est un objet (Lvalue) dont la valeur est égale à l'adresse d'un autre objet. On déclare un pointeur par l'instruction :

type **nom-du-pointeur*;

où *type* est le type de l'objet pointé. Cette déclaration déclare un identificateur, *nom-du-pointeur*, associé à un objet dont la valeur est l'adresse d'un autre objet de type *type*. L'identificateur *nom-du-pointeur* est donc en quelque sorte un identificateur d'adresse. Comme pour n'importe quelle Lvalue, sa valeur est modifiable.

Même si la valeur d'un pointeur est toujours un entier (éventuellement un entier long), le type d'un pointeur dépend du type de l'objet vers lequel il pointe. Cette distinction est indispensable à l'interprétation de la valeur d'un pointeur. En effet, pour un pointeur sur un objet de type char, la valeur donne l'adresse de l'octet où cet objet est stocké. Par contre, pour un pointeur sur un objet de type int, la valeur donne l'adresse du premier des 4 octets où l'objet est stocké. Dans l'exemple suivant, on définit un pointeur p qui pointe vers un entier i :

```
int i = 3;
int *p;

p = &i;
```

On se trouve dans la configuration

objet	adresse	valeur
i	4831836000	3

p	4831836004	4831836000

L'*opérateur unaire d'indirection* * permet d'accéder directement à la valeur de l'objet pointé. Ainsi, si p est un pointeur vers un entier i, *p désigne la valeur de i. Par exemple, le programme

```
main()
{
  int i = 3;
  int *p;

  p = &i;
  printf("*p = %d \n",*p);
}
```

imprime *p = 3.

Dans ce programme, les objets i et *p sont identiques : ils ont mêmes adresse et valeur. Nous sommes dans la configuration :

objet	adresse	valeur
i	4831836000	3
p	4831836004	4831836000
*p	4831836000	3

Cela signifie en particulier que toute modification de *p modifie i. Ainsi, si l'on ajoute l'instruction *p = 0; à la fin du programme précédent, la valeur de i devient nulle.

On peut donc dans un programme manipuler à la fois les objets p et *p. Ces deux manipulations sont très différentes. Comparons par exemple les deux programmes suivants :

```
main()
{
  int i = 3, j = 6;
  int *p1, *p2;
  p1 = &i;
  p2 = &j;
  *p1 = *p2;
}
```

et

```
main()
```

```
{
  int i = 3, j = 6;
  int *p1, *p2;
  p1 = &i;
  p2 = &j;
  p1 = p2;
}
```

Avant la dernière affectation de chacun de ces programmes, on est dans une configuration du type :

objet	adresse	valeur
i	4831836000	3
j	4831836004	6
p1	4831835984	4831836000
p2	4831835992	4831836004

Après l'affectation *p1 = *p2; du premier programme, on a

objet	adresse	valeur
i	4831836000	6
j	4831836004	6
p1	4831835984	4831836000
p2	4831835992	4831836004

Par contre, l'affectation p1 = p2 du second programme, conduit à la situation :

objet	adresse	valeur
i	4831836000	3
j	4831836004	6
p1	4831835984	4831836004
p2	4831835992	4831836004

II.2 Arithmétique des pointeurs

La valeur d'un pointeur étant un entier, on peut lui appliquer un certain nombre d'opérateurs arithmétiques classiques. Les seules opérations arithmétiques valides sur les pointeurs sont :

- l'addition d'un entier à un pointeur. Le résultat est un pointeur de même type que le pointeur de départ ;
- la soustraction d'un entier à un pointeur. Le résultat est un pointeur de même type que le pointeur de départ ;
- la différence de deux pointeurs pointant tous deux vers des objets de même type. Le résultat est un entier.

Notons que la somme de deux pointeurs n'est pas autorisée.

Si i est un entier et p est un pointeur sur un objet de type *type*, l'expression p + i désigne un pointeur sur un objet de type *type* dont la valeur est égale à la valeur de p incrémentée de i * sizeof(*type*). Il en va de même pour la soustraction d'un entier à un pointeur, et pour les opérateurs d'incrémentation et de décrémentation ++ et --. Par exemple, le programme

```
main()
{
  int i = 3;
  int *p1, *p2;
  p1 = &i;
  p2 = p1 + 1;
  printf("p1 = %ld \t p2 = %ld\n",p1,p2);
}
```

affiche p1 = 4831835984 p2 = 4831835988.

Par contre, le même programme avec des pointeurs sur des objets de type double :

```
main()
{
  double i = 3;
  double *p1, *p2;
  p1 = &i;
  p2 = p1 + 1;
  printf("p1 = %ld \t p2 = %ld\n",p1,p2);
}
```

affiche p1 = 4831835984 p2 = 4831835992.

Les opérateurs de comparaison sont également applicables aux pointeurs, à condition de comparer des pointeurs qui pointent vers des objets de même type.

L'utilisation des opérations arithmétiques sur les pointeurs est particulièrement utile pour parcourir des

tableaux. Ainsi, le programme suivant imprime les éléments du tableau tab dans l'ordre croissant puis décroissant des indices.

```
#define N 5

int tab[5] = {1, 2, 6, 0, 7};

main()

{

 int *p;

 printf("\n ordre croissant:\n");

 for (p = &tab[0]; p <= &tab[N-1]; p++)

   printf(" %d \n",*p);

 printf("\n ordre decroissant:\n");

 for (p = &tab[N-1]; p >= &tab[0]; p--)

   printf(" %d \n",*p);

}
```

Si p et q sont deux pointeurs sur des objets de type *type*, l'expression p - q désigne un entier dont la valeur est égale à (p - q)/sizeof(*type*) .

II.3 Allocation dynamique

Avant de manipuler un pointeur, et notamment de lui appliquer l'opérateur d'indirection *, il faut l'initialiser. Sinon, par défaut, la valeur du pointeur est égale à une constante symbolique notée NULL définie dans stdio.h. En général, cette constante vaut 0. Le test p == NULL permet de savoir si le pointeur p pointe vers un objet.

On peut initialiser un pointeur p par une affectation sur p. Par exemple, on peut affecter à p l'adresse d'une autre variable. Il est également possible d'affecter directement une valeur à *p. Mais pour cela, il faut d'abord réserver à *p un espace-mémoire de taille adéquate. L'adresse de cet espace-mémoire sera la valeur de p. Cette opération consistant à réserver un espace-mémoire pour stocker l'objet pointé s'appelle *allocation dynamique*. Elle se fait en C par la fonction malloc de la librairie standard stdlib.h. Sa syntaxe est

malloc(*nombre-octets*)

Cette fonction retourne un pointeur de type char * pointant vers un objet de taille *nombreoctets* octets. Pour initialiser des pointeurs vers des objets qui ne sont pas de type char, il faut convertir le type de la sortie de la fonction malloc à l'aide d'un cast. L'argument *nombre-octets* est souvent donné à l'aide de la fonction sizeof() qui renvoie le nombre d'octets utilisés pour stocker un objet.

Ainsi, pour initialiser un pointeur vers un entier, on écrit :

```
#include <stdlib.h>

int *p;

p = (int*)malloc(sizeof(int));
```

On aurait pu écrire également
p = (int*)malloc(4);
puisqu'un objet de type int est stocké sur 4 octets. Mais on préférera la première écriture qui a l'avantage d'être portable.

Le programme suivant

```
#include <stdio.h>

#include <stdlib.h>

main()

{

  int i = 3;

  int *p;

  printf("valeur de p avant initialisation = %ld\n",p);

  p = (int*)malloc(sizeof(int));

  printf("valeur de p apres initialisation = %ld\n",p);

  *p = i;

  printf("valeur de *p = %d\n",*p);

}
```

définit un pointeur p sur un objet *p de type int, et affecte à *p la valeur de la variable i. Il imprime à l'écran :

valeur de p avant initialisation = 0

valeur de p apres initialisation = 5368711424

valeur de *p = 3

Avant l'allocation dynamique, on se trouve dans la configuration

objet	adresse	valeur
i	4831836000	3
p	4831836004	0

A ce stade, *p n'a aucun sens. En particulier, toute manipulation de la variable *p générerait une violation mémoire, détectable à l'exécution par le message d'erreur Segmentation fault.

L'allocation dynamique a pour résultat d'attribuer une valeur à p et de réserver à cette adresse un espace-mémoire composé de 4 octets pour stocker la valeur de *p. On a alors

objet	adresse	valeur
i	4831836000	3
p	4831836004	5368711424
*p	5368711424	? (int)

*p est maintenant définie mais sa valeur n'est pas initialisée. Cela signifie que *p peut valoir n'importe quel entier (celui qui se trouvait précédemment à cette adresse). L'affectation *p = i; a enfin pour résultat d'affecter à *p la valeur de i. A la fin du programme, on a donc

objet	adresse	valeur
i	4831836000	3
p	4831836004	5368711424
*p	5368711424	3

Il est important de comparer le programme précédent avec

```
main()
{
  int i = 3;
  int *p;

  p = &i;
}
```

qui correspond à la situation

objet	adresse	valeur
i	4831836000	3
p	4831836004	4831836000
*p	4831836000	3

Dans ce dernier cas, les variables i et *p sont identiques (elles ont la même adresse) ce qui implique que toute modification de l'une modifie l'autre. Ceci n'était pas vrai dans l'exemple précédent où *p et iavaient la même valeur mais des adresses différentes.

On remarquera que le dernier programme ne nécessite pas d'allocation dynamique puisque l'espace-

mémoire à l'adresse &i est déjà réservé pour un entier.

La fonction malloc permet également d'allouer un espace pour plusieurs objets contigus en mémoire. On peut écrire par exemple

```
#include <stdio.h>

#include <stdlib.h>

main()

{

  int i = 3;

  int j = 6;

  int *p;

  p = (int*)malloc(2 * sizeof(int));

  *p = i;

  *(p + 1) = j;

  printf("p = %ld \t *p = %d \t p+1 = %ld \t *(p+1) = %d \n",p,*p,p+1,*(p+1));

}
```

On a ainsi réservé, à l'adresse donnée par la valeur de p, 8 octets en mémoire, qui permettent de stocker 2 objets de type int. Le programme affiche
p = 5368711424 *p = 3 p+1 = 5368711428 *(p+1) = 6 .

La fonction calloc de la librairie stdlib.h a le même rôle que la fonction malloc mais elle initialise en plus l'objet pointé *p à zéro. Sa syntaxe est

calloc(*nb-objets*,*taille-objets*)

Ainsi, si p est de type int*, l'instruction

```
p = (int*)calloc(N,sizeof(int));
```

est strictement équivalente à

```
p = (int*)malloc(N * sizeof(int));

for (i = 0; i < N; i++)

  *(p + i) = 0;
```

L'emploi de calloc est simplement plus rapide.

Enfin, lorsque l'on n'a plus besoin de l'espace-mémoire alloué dynamiquement (c'est-à-dire quand on n'utilise plus le pointeur p), il faut libérer cette place en mémoire. Ceci se fait à l'aide de l'instruction free qui a pour syntaxe

free(*nom-du-pointeur*);

A toute instruction de type malloc ou calloc doit être associée une instruction de type free.

POINTS IMPORTANTS A RETENIR

-Utilisation de l'opérateur d'adresse &
-Utilisations des pointeurs
- Arithmétique des pointeurs

CHAPITRE 4 : LES TABLEAUX ET LES CHAINES DE CARACTERES

I-LES TABLEAUX DE NOMBRES (INT ou FLOAT)

Les tableaux correspondent aux matrices en mathématiques. Un tableau est caractérisé par sa taille et par ses éléments.

Les tableaux à une dimension:

Déclaration: **type nom[dim];** Exemples: **int compteur[10];**

float nombre[20];

Cette déclaration signifie **que le compilateur réserve dim places en mémoire pour ranger les éléments du tableau.**

Exemples:

int compteur[10]; le compilateur réserve des places pour 10 entiers, soit 20 octets en TURBOC et 40 octets en C standard.

float nombre[20]; le compilateur réserve des places pour 20 réels, soit 80 octets.

Remarque: dim est nécessairement une VALEUR NUMERIQUE. Ce ne peut être en aucun cas une combinaison des variables du programme.

Utilisation: Un élément du tableau est repéré par son indice. En langage C les tableaux commencent à l'indice 0. L'indice maximum est donc dim-1.

Appel: **nom[indice]**

Exemples: **compteur[2] = 5;**

nombre[i] = 6.789;

printf("%d",compteur[i]);

scanf("%f",&nombre[i]);

Tous les éléments d'un tableau (correspondant au dimensionnement maximum) ne sont pas forcément définis.

D'une façon générale, les tableaux consomment beaucoup de place en mémoire. On a donc intérêt à les dimensionner au plus juste.

Exercice VI_1: Saisir 10 réels, les ranger dans un tableau. Calculer et afficher la moyenne et l'écart-type.

Les tableaux à plusieurs dimensions:

Tableaux à deux dimensions:

Déclaration: **type nom[dim1][dim2];** Exemples: **int compteur[4][5];**

float nombre[2][10];

Utilisation: Un élément du tableau est repéré par ses indices. En langage C les tableaux commencent aux indices 0. Les indices maximum sont donc dim1-1, dim2-1.

Appel: **nom[indice1][indice2]**

Exemples: **compteur[2][4] = 5;**

nombre[i][j] = 6.789;

printf("%d",compteur[i][j]);

scanf("%f",&nombre[i][j]);

Tous les éléments d'un tableau (correspondant au dimensionnement maximum) ne sont pas forcément définis.

Exercice VI_2: Saisir une matrice d'entiers 2x2, calculer et afficher son déterminant.

Tableaux à plus de deux dimensions:

On procède de la même façon en ajoutant les éléments de dimensionnement ou les indices nécessaires.

-Initialisation des tableaux

On peut initialiser les tableaux au moment de leur déclaration:

Exemples:

int liste[10] = {1,2,4,8,16,32,64,128,256,528};

float nombre[4] = {2.67,5.98,-8,0.09};

int x[2][3] = {{1,5,7},{8,4,3}}; /* 2 lignes et 3 colonnes */

-Tableaux et pointeurs

En déclarant un tableau, on définit automatiquement un pointeur (on définit en fait l'adresse du premier élément du tableau).

Les tableaux à une dimension:

Les écritures suivantes sont équivalentes:

int *tableau;	**int tableau[10];**	**déclaration**
tableau = (int*)malloc(sizeof(int)*10);		
***tableau**	**tableau[0]**	**le 1er élément**
***(tableau+i)**	**tableau[i]**	**un autre élément**
tableau	**&tableau[0]**	**adresse du 1er élément**
(tableau + i)	**&(tableau[i])**	**adresse d'un autre élément**

Il en va de même avec un tableau de réels (float).

Remarque: La déclaration d'un tableau entraine automatiquement la réservation de places en mémoire. Ce n'est pas le cas lorsque l'on déclare un pointeur. Il faut alors utiliser une fonction d'allocation dynamique comme malloc.

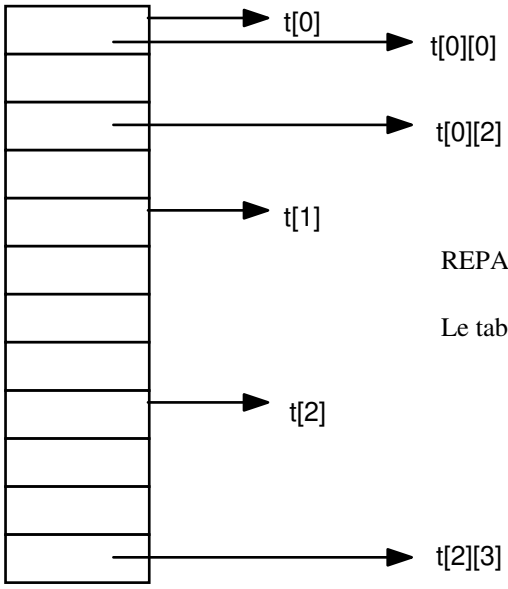
Les tableaux à plusieurs dimensions:

Un tableau à plusieurs dimensions est un pointeur de pointeur.

Exemple: **int t[3][4];** t est un pointeur de 3 tableaux de 4 éléments ou bien de 3 lignes à 4 éléments.

Les écritures suivantes sont équivalentes:

t[0]	**&t[0][0]**	**t**	**adresse du 1er élément**
t[1]	**&t[1][0]**		**adresse du 1er élément de la 2e ligne**
t[i]	**&t[i][0]**		**adresse du 1er élément de la ième ligne**
t[i]+1	**&(t[i][0])+1**		**adresse du 1er élément de la ième +1 ligne**



REPARTITION DES ELEMENTS EN MEMOIRE

Le tableau a été déclaréainsi: int t[3][4];

Exercice VI_3:

Un programme contient la déclaration suivante:

int tab[10] = {4,12,53,19,11,60,24,12,89,19};

Compléter ce programme de sorte d'afficher les adresses des éléments du tableau.

Exercice VI_4:

Un programme contient la déclaration suivante:

int tab[20] = {4,-2,-23,4,34,-67,8,9,-10,11, 4,12,-53,19,11,-60,24,12,89,19};

Compléter ce programme de sorte d'afficher les éléments du tableau avec la présentation suivante:

4	-2	-23	4	34
-67	8	9	-10	11
4	12	-53	19	11
-60	24	12	89	19

II-LES CHAINES DE CARACTERES

En langage C, les chaînes de caractères sont des **tableaux de caractères**. Leur manipulation est donc analogue à celle d'un tableau à une dimension:

Déclaration: **char nom[dim];** ou bien **char *nom;**

nom = (char*)malloc(dim);

Exemple: **char texte[dim];** ou bien **char *texte;**

texte = (char*)malloc(10);

Le compilateur réserve (dim-1) places en mémoire pour la chaîne de caractères: En effet, il ajoute toujours le caractère **NUL ('\0')** à la fin de la chaîne en mémoire.

Affichage à l'écran:

On peut utiliser la fonction **printf** et le format %s:

```
char texte[10] = "BONJOUR";
printf("VOICI LE TEXTE: %s\n",texte);
```

On utilisera si possible la fonction **puts** non formatée:

puts(texte); est équivalent à **printf("%s\n",texte);**

Saisie: On peut utiliser la fonction **scanf** et le format %s. Une chaîne étant un pointeur, on n'écrit pas le symbole &. On utilisera de préférence la fonction **gets** non formatée.

```
char texte[10];
printf("ENTRER UN TEXTE: ");
```

scanf("%s",texte); est équivalent à **gets(texte);**

Remarque: scanf ne permet pas la saisie d'une chaîne comportant des espaces: les caractères saisis à partir de l'espace ne sont pas pris en compte (l'espace est un délimiteur au même titre que LF) mais rangés dans le tampon d'entrée. Pour saisir une chaîne de type "il fait beau", il faut utiliser gets.

A l'issue de la saisie d'une chaîne de caractères, le compilateur ajoute '\0' en mémoire après le dernier caractère.

Comme expliqué au chapitre 2, gets et scanf utilisent le flux d'entrée.

Exercice VI_5:

Saisir une chaîne de caractères, afficher les éléments de la chaîne et leur adresse (y compris le dernier caractère '\0').

Exercice VI_6:

Saisir une chaîne de caractères. Afficher le nombre de e et d'espaces de cette chaîne.

Fonctions permettant la manipulation des chaînes:

Les bibliothèques fournies avec les compilateurs contiennent de nombreuses fonctions de traitement des chaînes de caractères. En BORLAND C++, elles appartiennent aux bibliothèques **string.h ou stdlib.h.** En voici quelques exemples:

Générales (string.h):

void *strcat(char *chaine1,char *chaine2) concatène les 2 chaînes, résultat dans chaine1, renvoie l'adresse de chaine1.

int strlen(char *chaine) renvoie la longueur de la chaine ('\0' non comptabilisé).

void *strrev(char *chaine) inverse la chaîne et, renvoie l'adresse de la chaine inversée.

Comparaison (string.h):

int strcmp(char *chaine1,char *chaine2) renvoie un nombre:

- positif si la chaîne1 est supérieure à la chaine2 (au sens de l'ordre alphabétique)
- négatif si la chaîne1 est inférieure à la chaîne2
- nul si les chaînes sont identiques.

Copie (string.h):

void *strcpy(char *chaine1,char *chaine2)

recopie chaine2 dans chaine1 et renvoie l'adresse de chaîne1.

Recopie (string.h):

Ces fonctions renvoient l'adresse de l'information recherchée en cas de succès, sinon le pointeur NULL (c'est à dire le pointeur de valeur 0 ou encore le pointeur faux).

void *strchr(chaine,caractère) recherche le caractère dans la chaîne.

void *strrchr(chaine,caractère) idem en commençant par la fin.

void *strstr(chaîne,sous-chaîne) recherche la sous-chaine dans la chaîne.

Conversions (stdlib.h):

int atoi(char *chaîne) convertit la chaîne en entier

float atof(char *chaine) convertit la chaîne en réel

exemple:
```
printf("ENTRER UN TEXTE: ");
gets(texte);
n = atoi(texte) ;
printf("%d",n); /* affiche 123 si texte vaut "123" */
                /* affiche 0 si texte vaut "bonjour" */
```

void *itoa(int n,char *chaîne,int base) convertit un entier en chaîne:

base: base dans laquelle est exprimé le nombre,

cette fonction renvoie l'adresse de la chaîne.

exemple: itoa(12,texte,10); texte vaut "12"

Pour tous ces exemples, la notation void signifie que la fonction renvoie un pointeur (l'adresse de l'information recherchée), mais que ce pointeur* ***n'est pas typé.*** *On peut ensuite le typer à l'aide de l'opérateur cast .*

Exemple:

```
int *adr;
char texte[10] = "BONJOUR";
adr = (int*)strchr(texte,'O');
```

Exercice VI_7:

L'utilisateur saisit le nom d'un fichier. Le programme vérifie que celui-ci possède l'extension .PAS

Exercice VI_8:

Un oscilloscope à mémoire programmable connecté à un PC renvoie l'information suivante sous forme d'une chaîne de caractères terminée par '\0'au PC:

"CHANNELA 0 10 20 30 40 30 20 10 0 -10 -20 -30 -40 -30 -20 -10 -0"

Afficher sur l'écran la valeur des points vus comme des entiers. On simulera la présence de l'oscilloscope en initialisant une chaîne de caractères char mesures[100].

POINTS IMPORTANTS A RETENIR

- Utilisations les tableaux de nombres (int ou float)
-Tableaux et pointeurs
- Utilisation des chaines de caractères

CHAPITRE 5 : LES FONCTIONS

En langage C les sous-programmes s'appellent des **fonctions.**

L'imbrication de fonctions n'est pas autorisée en C: une fonction ne peut pas être déclarée à l'intérieur d'une autre fonction. Par contre, une fonction peut **appeler** une autre fonction. Cette dernière doit être déclarée **avant** celle qui l'appelle.

Une fonction possède un et un seul point d'entrée, mais éventuellement plusieurs points de sortie (à l'aide du mot **return**).

Une variable connue uniquement d'une fonction ou de main() est une variable locale.

Une variable connue de tout le programme est une variable globale.

I-FONCTIONS SANS PASSAGE D'ARGUMENTS ET NE RENVOYANT RIEN AU PROGRAMME.

Une fonction ne renvoyant rien au programme est une fonction de type **void.**

Exemples et Exercices:

Exercice VII_1:

```
#include <stdio.h>
#include <conio.h>
#include "c:\bc5\teach_c\chap7\chap7.h"   /* fichier d'en-tete */

void bonjour()       /* declaration de la fonction */
```

```
{
printf("bonjour\n");
}

void main()              /* programme principal */
{
bonjour();   /* appel de la fonction */
printf("POUR CONTINUER FRAPPER UNE TOUCHE: ");
getch();
}
```

Le fichier d'en-tête est un fichier texte contenant uniquement la ligne suivante :

void bonjour(void) ;

c'est à dire la déclaration de la fonction bonjour, sous la forme de son prototype.

Ce fichier est lu par le compilateur, qui vérifie si l'on appelle correctement la fonction.

Exercice VII_2:

```
#include <stdio.h>
#include <conio.h>
#include "c:\bc5\teach_c\chap7\chap7.h"   /* fichier d'en-tete */

void bonjour()       /* declaration de la fonction */
{
printf("bonjour\n");
}

void coucou()        /* declaration de la fonction */
{
bonjour();       /* appel d'une fonction dans une fonction */
printf("coucou\n");
```

```
}

void main()             /* programme principal */
{
coucou();   /* appel de la fonction */
printf("POUR CONTINUER FRAPPER UNE TOUCHE: ");
getch();
}
```

Le fichier d'en-tête contient maintenant les lignes suivantes :

```
void bonjour(void) ;
void coucou(void) ;
```

Exercice VII_3:

```
#include <stdio.h>
#include <conio.h>
#include "c:\bc5\courc_C\teach_c\chap7\chap7.h"   /* fichier d'en-tete */

void carre()       /* declaration de la fonction */
{
int n,n2;                   /* variables locales à carre */
printf("ENTRER UN NOMBRE: ");
scanf("%d",&n);
n2 = n*n ;
printf("VOICI SON CARRE: %d\n",n2);
}


void main()             /* programme principal */
{
carre();   /* appel de la fonction */
```

```
printf("POUR CONTINUER FRAPPER UNE TOUCHE: ");
getch();
}
```

Les variables n et n2 ne sont connues que de la fonction carré.

Le fichier d'en-tête contient maintenant les lignes suivantes :

```
void bonjour(void) ;
void coucou(void) ;
void carre(void) ;
```

Les déclarations inutiles sont ignorées.

Exercice VII_4:

```
#include <stdio.h>
#include <conio.h>
#include "c:\bc5\courc_C\teach_c\chap7\chap7.h"   /* fichier d'en-tete */

void carre()       /* declaration de la fonction */
{
int n, n2;                  /* variables locales a carre */
printf("ENTRER UN NOMBRE: ");
scanf("%d",&n);
n2 = n*n;
printf("VOICI SON CARRE: %d\n",n2);
}

void cube()       /* declaration de la fonction */
{
int n, n3;                  /* variables locales a cube */
printf("ENTRER UN NOMBRE: ");
scanf("%d",&n);
```

```
n3 = n*n*n;
printf("VOICI SON CUBE: %d\n",n3);
}

void main()             /* programme principal */
{
char choix;                 /* variable locale a main() */
printf("CALCUL DU CARRE TAPER 2\n");
printf("CALCUL DU CUBE TAPER 3\n");
printf("\nVOTRE CHOIX: ");
scanf("%c",&choix);
```

```
switch(choix)
        {
        case '2':carre();break;
        case '3':cube();break;
        }
printf("\nPOUR CONTINUER FRAPPER UNE TOUCHE: ");
getch();
}
```

Les 2 variables locales n sont indépendantes l'une de l'autre.

La variable locale choix n'est connue que de main().

Le fichier d'en-tête contient maintenant les lignes suivantes :

```
void bonjour(void) ;
void coucou(void) ;
void carre(void) ;
void cube(void) ;
```

Exercice VII_5:

```
#include <stdio.h>
#include <conio.h>
#include "c:\bc5\courc_C\teach_c\chap7\chap7.h"   /* fichier d'en-tete */

int n;              /* variable globale, connue de tout le programme */

void carre()       /* declaration de la fonction */
{
int n2;   /* variable locale */
printf("ENTRER UN NOMBRE: ");
scanf("%d",&n);
n2 = n*n;
printf("VOICI SON CARRE: %d\n",n2);
```

```
}

void cube()       /* declaration de la fonction */
{
int n3;  /* variable locale */
printf("ENTRER UN NOMBRE: ");
scanf("%d",&n);
n3 = n*n*n;
printf("VOICI SON CUBE: %d\n",n3);
}

void main()             /* programme principal */
{
char choix;                /* variable locale a main() */
printf("CALCUL DU CARRE TAPER 2\n");
printf("CALCUL DU CUBE TAPER 3\n");
printf("\nVOTRE CHOIX: ");
scanf("%c",&choix);
switch(choix)
        {
        case '2':carre();break;
        case '3':cube();break;
        }
printf("\nPOUR CONTINUER FRAPPER UNE TOUCHE: ");
getch();
}
```

La variable globale n est connue de tout le programme (fonctions et main()).

La variable locale choix n'est connue que de main().

Le fichier d'en-tête n'a pas changé.

Un programme bien construit possède peu de variables globales.

Exercice VII_6:

Un programme contient la déclaration suivante:

int tab[10] = {1,2,4,8,16,32,64,128,256,512}; /* variable globale */

Ecrire une fonction de prototype **void affiche(void)** qui affiche les éléments du tableau, et leur adresse; la mettre en oeuvre dans main().

Actualiser le fichier d'en-tête en conséquence.

II-FONCTION RENVOYANT UNE VALEUR AU PROGRAMME ET SANS PASSAGE D'ARGUMENTS

Dans ce cas, la fonction, après exécution, renvoie une valeur. Le type de cette valeur est déclaré avec la fonction. La valeur retournée est spécifiée à l'aide du mot réservé *return*.

Exemple et Exercice VII_7:

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <time.h>
#include "c:\bc5\courc_C\teach_c\chap7\chap7.h"   /* fichier d'en-tete */

int lance_de()  /* declaration de la fonction */
{
int test;  /* variable locale */
test = random(6) + 1;
return(test);
}

void main()
```

```
{
int resultat;
randomize();
resultat = lance_de();
     /* resultat prend la valeur retournee par le sous-programme */
printf("Vous avez obtenu le nombre: %d\n",resultat);
printf("POUR SORTIR FRAPPER UNE TOUCHE ");
getch();
}
```

Il faut ajouter au fichier d'en-tête la ligne suivante :

```
int lance_de(void) ;
```

Exercice VII_8:

Un programme contient la déclaration suivante:

```
float liste[8] = {1.6,-6,9.67,5.90,345,-23.6,78,34.6};      /* variable globale */
```

Ecrire une fonction de prototype **float min(void)** qui renvoie le minimum de la liste.

Ecrire une fonction de prototype **float max(void)** qui renvoie le maximum de la liste.

Les mettre en oeuvre dans main().

Actualiser le fichier d'en-tête en conséquence.

III-FONCTIONS AVEC PASSAGE D'ARGUMENTS

Ce paragraphe traite directement du cas général: fonctions de type void ou non (renvoyant une valeur ou non).

Ces fonctions utilisent les valeurs de certaines variables du programme les ayant appelé: on passe ces valeurs au moyen d'arguments déclarés avec la fonction.

Exemple et Exercice VII_9:

```
#include <stdio.h>
#include <conio.h>
#include "c:\bc5\courc_C\teach_c\chap7\chap7.h"   /* fichier d'en-tete */

int carre(int x)  /* declaration de la fonction */
{               /* x est un parametre*/
int x2;  /* variable locale */
x2 = x*x;
return(x2);
}
void main()
{
int n1, n2, res1, res2;              /* variables locales */
printf("ENTRER UN NOMBRE: ");
scanf("%d",&n1);
res1 = carre(n1);
printf("ENTRER UN AUTRE NOMBRE: ");
scanf("%d",&n2);
res2 = carre(n2);
printf("VOICI LEURS CARRES: %d   %d\n\n",res1, res2);
printf("POUR SORTIR FRAPPER UNE TOUCHE: ");
getch();
}
```

Dans le fichier d'en-tête, le prototype de la fonction carre devient **int carre(int) ;**

L'expression ***int carre(int)*** est appelée « prototype réduit » de la fonction.

L'expression ***int carre(int x)*** est appelée « prototype complet » de la fonction.

On peut ainsi appeler la fonction carre autant de fois que l'on veut avec des variables différentes.

x est un paramètre, ou argument: ce n'est pas une variable du programme.

S'il y a plusieurs arguments à passer, il faut respecter la syntaxe suivante:

Exemples: void fonction1(int x,int y) void fonction2(int a,float b,char c)

Exercice VII_10:

Ecrire une fonction de prototype **int puissance(int a, int b)** qui calcule a^b, a et b sont des entiers (cette fonction n'existe pas en bibliothèque). La mettre en oeuvre dans main(). Actualiser le fichier d'en-tête en conséquence.

Exercice VII_11:

tab1 et tab2 sont des variables locales à main:

int tab1[10] = {0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19};

int tab2[10] = {-19,18,-17,16,-15,14,-13,12,-11,10,-9,8,-7,6,-5,4,-3,2,-1,0};

Ecrire une fonction de prototype **void affiche(int *tab)** qui permet d'afficher les 20 nombres suivant un tableau 4x5.

La mettre en oeuvre dans main() pour afficher tab1 et tab2.

Il faut ici ajouter la ligne **void affiche(int *) ;** dans le fichier d'en-tête.

On dit, dans ce cas, que l'on a passé le paramètre PAR ADRESSE.

NB : **Résumé sur variables et fonctions**

On a donc vu qu'une variable **globale** est déclarée au début du programme et qu'elle est connue de tout le programme. **Les variables globales sont initialisées à 0 au début de l'exécution du programme, sauf si on les initialise à une autre valeur.**

On a vu aussi qu'une variable **locale** (déclarée au début d'une fonction ou de main()) n'est connue que de cette fonction ou de main(). Une variable locale est encore appelée **automatique.**

Les variables locales ne sont pas initialisées (sauf si on le fait dans le programme) et elles perdent leur valeur à chaque appel à la fonction.

On peut allonger la durée de vie d'une variable locale en la déclarant **static.** Lors d'un nouvel appel à la fonction, la variable garde la valeur obtenue à la fin de l'exécution précédente. Une variable **static** est initialisée à 0 lors du premier appel à la fonction.

Exemple: **int i;** devient **static int i;**

Exercice VII_12:

Quelle sera la valeur finale de n si i est déclarée comme variable static, puis comme variable automatique ?

```
#include <stdio.h>
#include <conio.h>
#include "c:\bc5\courc_C\teach_c\chap7\chap7.h"   /* fichier d'en-tete */

int n;   /* initialisee … 0 */

void calcul()
{
static int i;            /* initialisee … 0 */
i++;
printf("i=%d\n",i);
n = n+i;
}

void main()
{
calcul();
printf("n= %d\n",n);
calcul();
printf("n= %d\n",n);
printf("\nPOUR SORTIR FRAPPER UNE TOUCHE ");
getch();
```

}

LE PASSAGE DE PARAMETRES ENTRE FONCTIONS
OU ENTRE FONCTIONS ET PROGRAMME PRINCIPAL

En langage C, le passage de paramètre se fait uniquement par adresse. Autrement dit, **une fonction ne peut pas modifier la valeur des variables locales à main() ou à une autre fonction. Elle ne peut modifier que le contenu de l'adresse de cette variable.**

Exemple: Fonction permettant d'échanger la valeur de 2 variables:

Syntaxe **qui conduit à une erreur:**

```
#include <stdio.h>
#include "c:\bc5\courc_C\teach_c\chap7\chap7.h"
void ech(int x,int y)
{
int tampon;
tampon = x;
x = y;
y = tampon;
}

void main()
{
int a = 5 , b = 8;
ech(a,b);
printf("a=%d\n" , a) ;
printf(" b=%d\n ", b) ;
}
```

PASSAGE DES PARAMETRES PAR VALEUR

Syntaxe **correcte:**

```
#include <stdio.h>

void ech(int *x,int *y)
{
int tampon;
tampon = *x;
*x = *y;
*y = tampon;
}

void main()
{
int a = 5 , b = 8 ;
ech(&a,&b);
printf("a=%d\n ", a) ;
printf("b=%d\n", b) ;
}
```

PASSAGE DES PARAMETRES PAR ADRESSE

a et b sont des variables locales à main(). La fonction ech ne peut donc pas modifier leur valeur. On le fait donc en passant par l'adresse de ces variables.

Expérimenter ces deux exemples, noter les résultats obtenus.

Dans un deuxième temps, afficher dans main() les adresses de a et de b et dans ech les adresses **de** x et **de** y (programme de gauche), les adresses x et y (programme de droite).

Le problème ne se pose pas lorsque le paramètre est un pointeur ou un tableau.

Exercice VII_13:

Saisir les 3 couleurs d'une résistance, afficher sa valeur.

Une fonction de prototype **float conversion(char *couleur)** calcule le nombre associé à chaque couleur.

Exercice VII_14:

Calculer et afficher les racines de $ax^2+bx+c=0$.

Une fonction de prototype **void saisie(float *aa,float *bb,float *cc)** permet de saisir a,b,c.

Une fonction de prototype **void calcul(float aa,float bb,float cc)** exécute les calculs et affiche les résultats.

a, b, c sont des variables locales à main();

main() se contente d'appeler **saisie(&a,&b,&c) et calcul(a,b,c).**

Exercice VII_15:

Ecrire une fonction de prototype **void saisie(int *tx)** qui saisie des entiers (au maximum 20), le dernier est 13. Cette fonction renvoie au programme principal **adr_debut et adr_fin,** les adresses respectives du 1er nombre et du dernier nombre saisis.

- **adr_debut** et **adr_fin** sont des variables **globales** à tout le programme.

- Le programme principal appelle **saisie(tab)** et affiche la valeur de **adr_debut** et **adr_fin** en hexadécimal; tab est une variable locale à main().

Exercice VII_16:

Modifier la fonction de prototype **void affiche(int *tx)** de l'exercice VII_11 de sorte d'afficher les nombres en tableau 4x5 mais en s'arrêtant à **adr_fin.** Compléter le programme principal de l'exercice VII_15 par un appel à **affiche(tab).**

POINTS IMPORTANTS A RETENIR

- Fonctions sans passage d'arguments et ne renvoyant rien au programme.
- Fonctions avec passage d'arguments
- Le passage de paramètres entre fonctions ou entre fonctions et programme principal

CHAPITRE 6 : LES TYPES DE VARIABLES COMPLEXES

I. LES DECLARATIONS DE TYPE SYNONYMES: TYPEDEF

On a vu les types de variables utilisés par le langage C: char, int, float, pointeur; le chapitre 7 traitera des fichiers (type FILE).

Le programmeur a la possibilité de créer ses propres types: **Il suffit de les déclarer en début de programme** (après les déclarations des bibliothèques et les « define ») avec la syntaxe suivante:

Exemples:

```
typedef int entier;                /* on définit un nouveau type "entier" synonyme de "int" */

typedef int vecteur[3];   /* on définit un nouveau type "vecteur" synonyme */
                                   /* de "tableau de 3 entiers"        */

typedef float *fpointeur;          /* on définit un nouveau type "fpointeur" synonyme */
                                   /* de "pointeur sur un réel */
```

Utilisation: La portée de la déclaration de type dépend de l'endroit où elle est déclarée: dans main(), le type n'est connu que de main(); en début de programme, le type est reconnu dans tout le programme.

```
#include <stdio.h>

typedef  int entier;
typedef float point[2];

void main()
{
```

```
entier n = 6;
point xy;
xy[0] = 8.6;
xy[1] = -9.45;

etc ...

}
```

Exercice VIII_1: Afficher la taille mémoire d'un point à l'aide de l'opérateur sizeof.

Exercice VIII_2: Définir le type **typedef char ligne[80];**

Déclarer dans main() un pointeur de ligne; lui attribuer de la place en mémoire (pour 5 lignes). Ecrire une fonction qui effectue la saisie de 5 lignes puis une autre qui les affiche. Les mettre en oeuvre dans main().

Attention, le fichier d'en-tête doit contenir la déclaration du nouveau type.

II. LES STRUCTURES

Le langage C autorise la déclaration de types particuliers: les structures . Une structure est constituée de plusieurs éléments de même type ou non.

Exemple:

Déclaration:

```
typedef struct          /* On définit un type struct */
{
char nom[10];
char prenom[10];
int age;
```

float note;

}

fiche;

Utilisation:

On déclare des variables par exemple: **fiche f1,f2;**

puis, par exemple: **strcpy(f1.nom,"DUPONT");**

strcpy(f1.prenom,"JEAN");

f1.age = 20;

f1.note = 11.5;

L'affectation globale est possible avec les structures: on peut écrire: **f2 = f1;**

Exercice VIII_3: Créer la structure ci-dessus, saisir une fiche, l'afficher.

III. STRUCTURES ET TABLEAUX

On peut définir un tableau de structures (mais ceci est assez peu utilisé) :

Exemple: (à partir de la structure définie précédemment)

Déclaration: **fiche f[10]; /* on déclare un tableau de 10 fiches */**

Utilisation: **strcpy(f[i].nom,"DUPONT") /* pour un indice i quelconque */**

```
strcpy(f[i].prenom,"JEAN");
f[i].age = 20;
f[i].note = 11.5;
```

Exercice VIII_4: Créer une structure **point{int num;float x;float y;}**

Saisir 4 points, les ranger dans un tableau puis les afficher.

IV. STRUCTURES ET POINTEURS

On peut déclarer des pointeurs sur des structures. Cette syntaxe est très utilisée en langage C, **elle est notamment nécessaire lorsque la structure est un paramètres modifiable dans la fonction.**

Un symbole spécial a été créé pour les pointeurs de structures, il s'agit du symbole **->**

Exemple: (à partir de la structure définie précédemment)

Déclaration: **fiche *f; /* on déclare un pointeur de fiche */**

Utilisation:
```
f = (fiche*)malloc(sizeof(fiche)); /* réserve de la place */
strcpy(f->nom,"DUPONT");
strcpy(f->prenom,"JEAN");
f->age = 20;
f->note = 11.5;
```

Exercice VIII_5: Reprendre l'exercice VIII_2: Une fonction saisie permet de saisir un point, une fonction affiche permet d'afficher un point.

POINTS IMPORTANTS A RETENIR

- Les types de variables complexes.
-Les structures
-Les Structures et les tableaux
-Les structures et les pointeurs

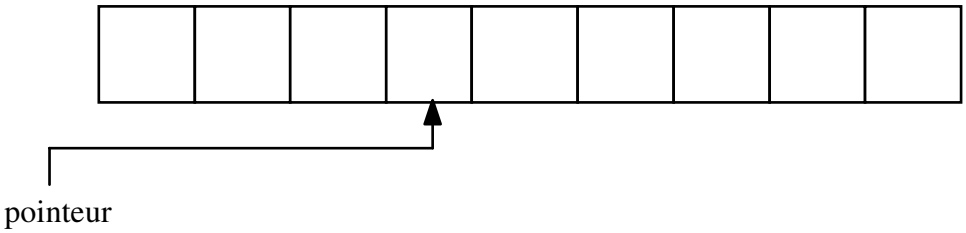
CHAPITRE 7 : LES FICHIERS

I-GENERALITES

Un fichier est un ensemble d'informations stockées sur une mémoire de masse (disque dur, disquette, bande magnétique, CD-ROM).

En C, un fichier est une suite d'octets. Les informations contenues dans le fichier ne sont pas forcément de même type (un char, un int, une structure ...)

Un **pointeur** fournit l'adresse d'une information quelconque.



On distingue généralement deux types d'accès:

1- Accès séquentiel (possible sur tout support, mais seul possible sur bande magnétique):

- Pas de cellule vide.
- On accède à une cellule quelconque en se déplaçant (via un pointeur), depuis la cellule de départ.
- On ne peut pas détruire une cellule.
- On peut par contre tronquer la fin du fichier.
- On peut ajouter une cellule à la fin.

2- Accès direct (RANDOM I/O) (Utilisé sur disques, disquettes, CD-ROM où l'accès séquentiel est possible aussi).

- Cellule vide possible.

- On peut directement accéder à une cellule.

- On peut modifier (voir détruire) n'importe quelle cellule.

Il existe d'autre part deux façons de coder les informations stockées dans un fichier :

1- En binaire :

Fichier dit « binaire », les informations sont codées telles que. Ce sont en général des fichiers de nombres. Ils ne sont pas listables.

2- en ASCII :

Fichier dit « texte », les informations sont codées en ASCII. Ces fichiers sont listables. Le dernier octet de ces fichiers est EOF (caractère ASCII spécifique).

II-MANIPULATION DES FICHIERS

Opérations possibles avec les fichiers: Créer - Ouvrir - Fermer - Lire - Ecrire - Détruire - Renommer. La plupart des fonctions permettant la manipulation des fichiers sont rangées dans la bibliothèque standard STDIO.H, certaines dans la bibliothèque IO.H pour le BORLAND C++.

Le langage C ne distingue pas les fichiers à accès séquentiel des fichiers à accès direct, certaines fonctions de la bibliothèque livrée avec le compilateur permettent l'accès direct. Les fonctions standards sont des fonctions d'accès séquentiel.

1 - Déclaration: **FILE *fichier; /* majuscules obligatoires pour FILE */**

On définit un pointeur. Il s'agit du pointeur représenté sur la figure du début de chapitre. Ce pointeur fournit l'adresse d'une cellule donnée.

La déclaration des fichiers doit figurer AVANT la déclaration des autres variables.

2 - Ouverture: **FILE *fopen(char *nom, char *mode);**

On passe donc 2 chaînes de caractères

nom: celui figurant sur le disque, exemple: "a :\toto.dat"

mode (pour les fichiers TEXTES) :

"r" lecture seule

"w" écriture seule (destruction de l'ancienne version si elle existe)

"w+" lecture/écriture (destruction ancienne version si elle existe)

" r+" lecture/écriture d'un fichier existant (mise à jour), pas de création d'une nouvelle version.

"a+" lecture/écriture d'un fichier existant (mise à jour), pas de création d'une nouvelle version, le pointeur est positionné à la fin du fichier.

mode (pour les fichiers BINAIRES) :

"rb" lecture seule

"wb" écriture seule (destruction de l'ancienne version si elle existe)

"wb+" lecture/écriture (destruction ancienne version si elle existe)

"rb+" lecture/écriture d'un fichier existant (mise à jour), pas de création d'une nouvelle version.

"ab+" lecture/écriture d'un fichier existant (mise à jour), pas de création d'une nouvelle version, le pointeur est positionné à la fin du fichier.

A l'ouverture, le pointeur est positionné au début du fichier (sauf "a+" et "ab+")

Exemple : **FILE *fichier ;**

fichier = fopen(" a :\toto.dat ", "rb") ;

3 - Fermeture: **int fclose(FILE *fichier);**

Retourne 0 si la fermeture s'est bien passée, EOF en cas d'erreur.

Il faut toujours fermer un fichier à la fin d'une session. mode (pour les fichiers TEXTE) :

Exemple : **FILE *fichier ;**

fichier = fopen(" a :\toto.dat", "rb") ;

/* Ici instructions de traitement */

fclose(fichier) ;

4 - Destruction: **int remove(char *nom);**

Retourne 0 si la fermeture s'est bien passée.

Exemple : **remove(" a :\toto.dat ") ;**

5 - Renommer: **int rename(char *oldname, char *newname);**

Retourne 0 si la fermeture s'est bien passée.

6 - Positionnement du pointeur au début du fichier: **void rewind(FILE *fichier);**

7- Ecriture dans le fichier:

int putc(char c, FILE *fichier);

Ecrit la valeur de c à la position courante du pointeur , le pointeur avance d'une case mémoire.

Retourne EOF en cas d'erreur.

Exemple : **putc('A', fichier) ;**

int putw(int n, FILE *fichier);

Idem, n de type int, le pointeur avance du nombre de cases correspondant à la taille d'un entier (4 cases en C standard).

Retourne n si l'écriture s'est bien passée.

int fputs(char *chaîne, FILE *fichier); idem avec une chaîne de caractères, le pointeur avance de la longueur de la chaine ('\0' n'est pas rangé dans le fichier).

Retourne EOF en cas d'erreur.

Exemple : **fputs(« BONJOUR ! », fichier) ;**

int fwrite(void *p,int taille_bloc,int nb_bloc,FILE *fichier); p de type pointeur, écrit à partir de la position courante du pointeur fichier nb_bloc X taille_bloc octets lus à partir de l'adresse p. Le pointeur fichier avance d'autant.

Le pointeur p est vu comme une adresse, son type est sans importance.

Retourne le nombre de blocs écrits.

Exemple: taille_bloc=4 (taille d'un entier en C), nb_bloc=3, écriture de 3 octets.

```
int tab[10] ;
fwrite(tab,4,3,fichier) ;
```

int fprintf(FILE *fichier, char *format, liste d'expressions); réservée plutôt aux fichiers ASCII.

Retourne EOF en cas d'erreur.

Exemples: **fprintf(fichier,"%s","il fait beau");**

fprintf(fichier,%d,n);

fprintf(fichier,"%s%d","il fait beau",n);

Le pointeur avance d'autant.

8 - Lecture du fichier:

int getc(FILE *fichier); lit 1 caractère, mais retourne un entier n; retourne EOF si erreur ou fin de fichier; le pointeur avance d'une case.

Exemple: **char c ;**

c = (char)getc(fichier) ;

int getw(FILE *fichier); idem avec un entier; le pointeur avance de la taille d'un entier.

Exemple: **int n ;**

n = getw(fichier) ;

char *fgets(char *chaine,int n,FILE *fichier); lit n-1 caractères à partir de la position du pointeur et les range dans chaine en ajoutant '\0'.

int fread(void *p,int taille_bloc,int nb_bloc,FILE *fichier); analogue à fwrite en lecture.

Retourne le nombre de blocs lus, et 0 à la fin du fichier.

int fscanf(FILE *fichier, char *format, liste d'adresses); analogue à fprintf en lecture.

9 - Gestion des erreurs:

fopen retourne le pointeur NULL si erreur (Exemple: impossibilité d'ouvrir le fichier).

fgets retourne le pointeur NULL en cas d'erreur ou si la fin du fichier est atteinte.

la fonction **int feof(FILE *fichier)** retourne 0 tant que la fin du fichier n'est pas atteinte.

la fonction **int ferror(FILE *fichier)** retourne 1 si une erreur est apparue lors d'une manipulation de fichier, 0 dans le cas contraire.

10 - Fonction particulière aux fichiers à acces direct:

int fseek(FILE *fichier,int offset,int direction) déplace le pointeur de offset cases à partir de direction.

Valeurs possibles pour direction:

0 -> à partir du début du fichier.

1 -> à partir de la position courante du pointeur.

2 -> en arrière, à partir de la fin du fichier.

Retourne 0 si le pointeur a pu être déplacé;

EXERCICES

Exercice IX_1: Copier un fichier texte dans un autre: créer un fichier "essai.dat" sous éditeur. Tester le programme en vérifiant après exécution la présence du fichier copié dans le directory.

Exercice IX_2: Calculer et afficher le nombres de caractères d'un fichier ASCII (Utiliser n'importe quel fichier du répertoire).

Exercice IX_3: Créer et relire un fichier binaire de 10 entiers.

Exercice IX_4: Lire un fichier texte, avec un contrôle d'erreur: L'utilisateur saisit le nom du fichier, la machine retourne le listing du fichier s'il existe et un message d'erreur s'il n'existe pas.

Exercice IX_5: Créer et relire un fichier texte de 5 chaînes de 3 caractères.

Exercice IX_6: Ajouter une fiche (c'est à dire une chaîne de 3 caractères) au fichier précédent et relire le fichier.

Exercice IX_7: Rechercher une fiche dans le fichier précédent.

Exercice IX_8: Exercice récapitulatif: Créer une structure nom, prénom, âge. Ecrire un programme de gestion de fichier (texte) avec menu d'accueil: possibilité de créer le fichier, de le lire, d'y ajouter une fiche, d'en rechercher une.

POINTS IMPORTANTS A RETENIR

-Les types d'accès au fichiers

-Manipulation des fichiers

BIBLIOGRAPHIE :

- Cours de programmation avancée, Le langage C, Université de Luxembourg , 2004-2005.
- Initiation à la programmation, université paris dauphine, iup génie mathématique et informatique, 2004-2005.
- Algorithmique et programmation : les bases (C), PAD – INPT, mai–juin 2006.
- Langage c, IUT de CACHAN.